MODULE 4 · LECTURE 20

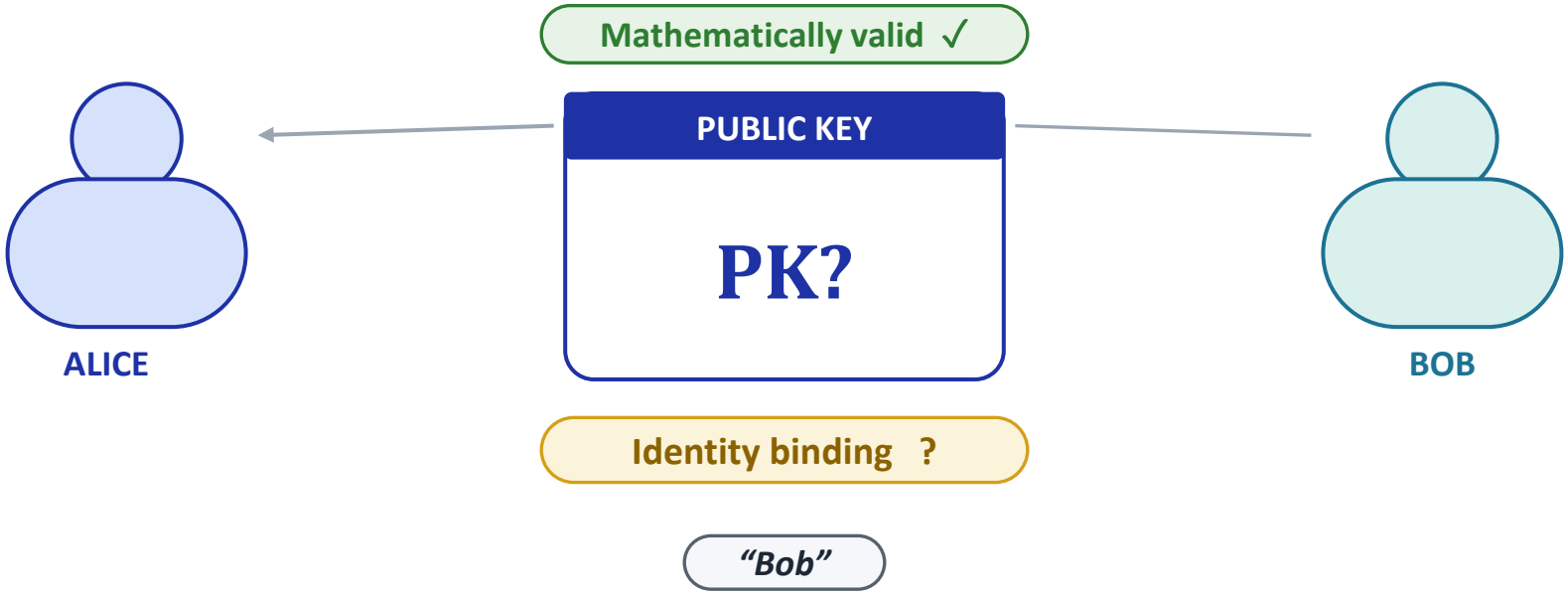
CS 6530 Applied Cryptography

Public-Key Distribution and Trust Models

Module 4 – Key Management, Certificates and PKI

THE TRUST PROBLEM

I Have a Public Key — But Whose Key Is It?



Claim: Bob ↔ PK? Evidence: ?

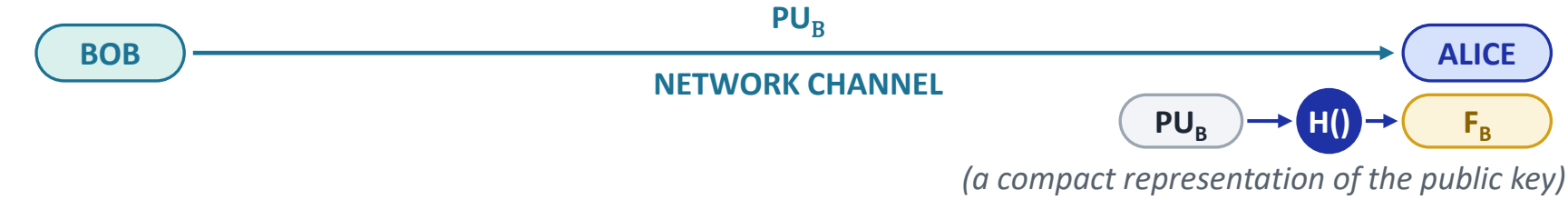
? THINK LIKE AN ENGINEER

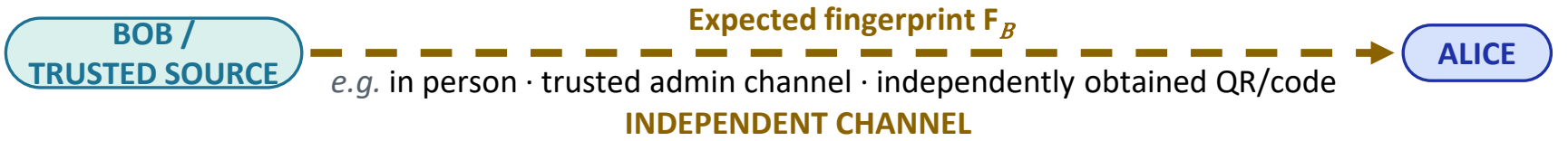
What additional evidence would convince Alice that this is really Bob's key?

The real problem is establishing a trusted binding: Bob ↔ PK_B.

TRUST PATH · Out of Band (OOB)

Verify the Key Through an Independent Channel





H(received PU_B) vs expected F_B
MATCH ✓ → Trust: Bob ↔ PU_B

THE FINGERPRINT DOES NOT CREATE TRUST
The independent verification channel does.

MALLORY → fake PU_B + fake fingerprint — same compromised channel
No independent evidence ✗

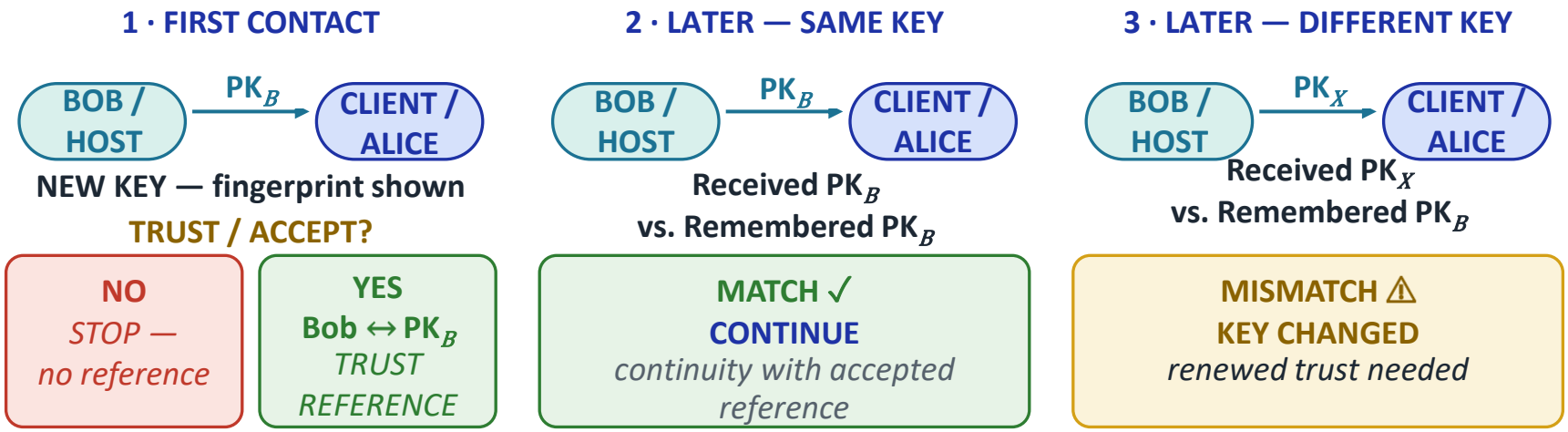
? THINK LIKE AN ENGINEER

If the public key and its fingerprint arrive through the same compromised channel, what has the fingerprint added?

A fingerprint authenticates a key binding only when its expected value is verified independently.

TRUST PATH · Trust On First Use (TOFU)

Trust the First Key — Then Remember It



ACCEPTANCE ≠ INDEPENDENT VERIFICATION

Accepting the first key authorizes it as the reference — it does not prove the binding was genuine.

MALLORY intercepts FIRST contact → PK_M presented as Bob's key → accepted → stored: Bob ↔ PK_M

A false first reference can persist.

First accepted binding becomes the trust reference; later observations are checked against it.

? THINK LIKE AN ENGINEER

What assurance do we really have about the very first accepted key?

First-use acceptance alone does not independently verify the original identity–key binding.

TOFU establishes trust from an accepted first binding and authenticates continuity against that remembered reference.

APPLIED CONNECTION · SSH

SSH Host Keys: TOFU in Everyday Use



HOST-KEY FINGERPRINT — SHA256: 3f:a1:9c:… (illustrative)
UNKNOWN HOST — authenticity not yet established.
TRUST / CONTINUE? yes / no

NO → stop, no binding stored

PATH A · PLAIN TOFU

Unknown host key → user accepts → store in known_hosts

accepted — not independently proven the key belongs to the host

YES → store in known_hosts (see below)

PATH B · OOB-STRENGTHENED FIRST CONTACT

Fingerprint shown → compare independently → MATCH ✓ → store

independently verified first binding — reconnects to the OOB slide

SAME KEY → ✓ CONTINUE
continuity with remembered host

CHANGED KEY → ⚠ WARNING
renewed trust needed

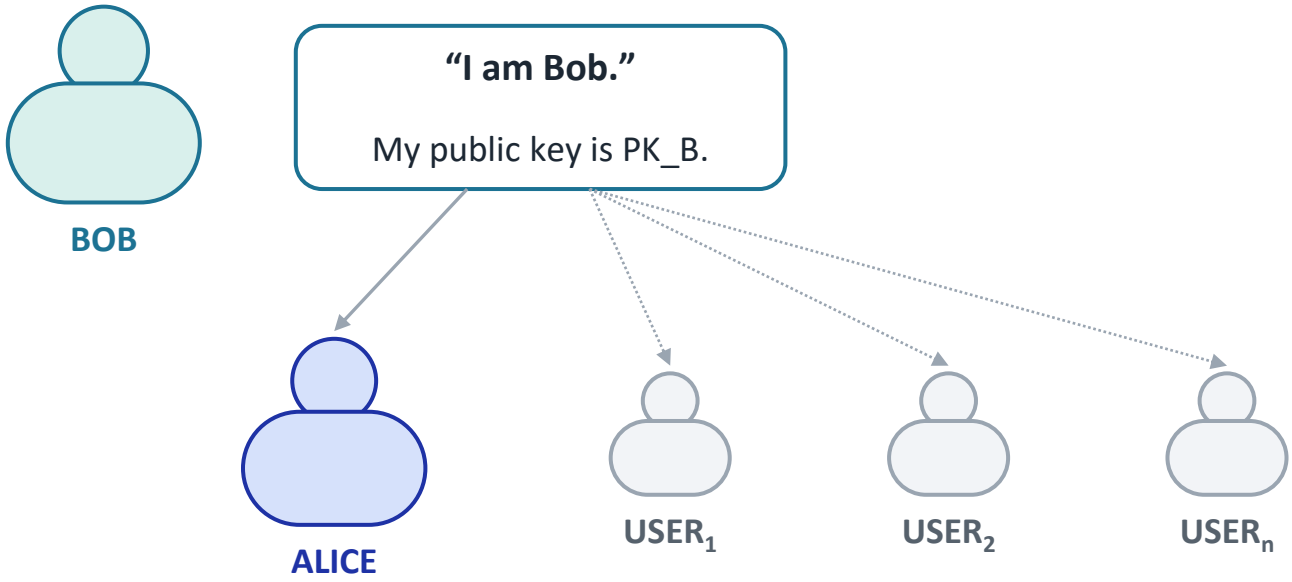
HOST KEY ≠ USER LOGIN KEY
The host key authenticates the server; a user key may separately authenticate the user.

? THINK LIKE AN ENGINEER
When a legitimate SSH host key changes, what must the user/client establish again?

SSH remembers an accepted host-key binding for continuity; independent fingerprint verification can strengthen first contact.

PUBLIC-KEY DISTRIBUTION · ANNOUNCEMENT

“Here Is My Public Key.”



website · email/message · public post · key server

Simple ✓ | **No intermediary ✓** | **Easy to distribute ✓**

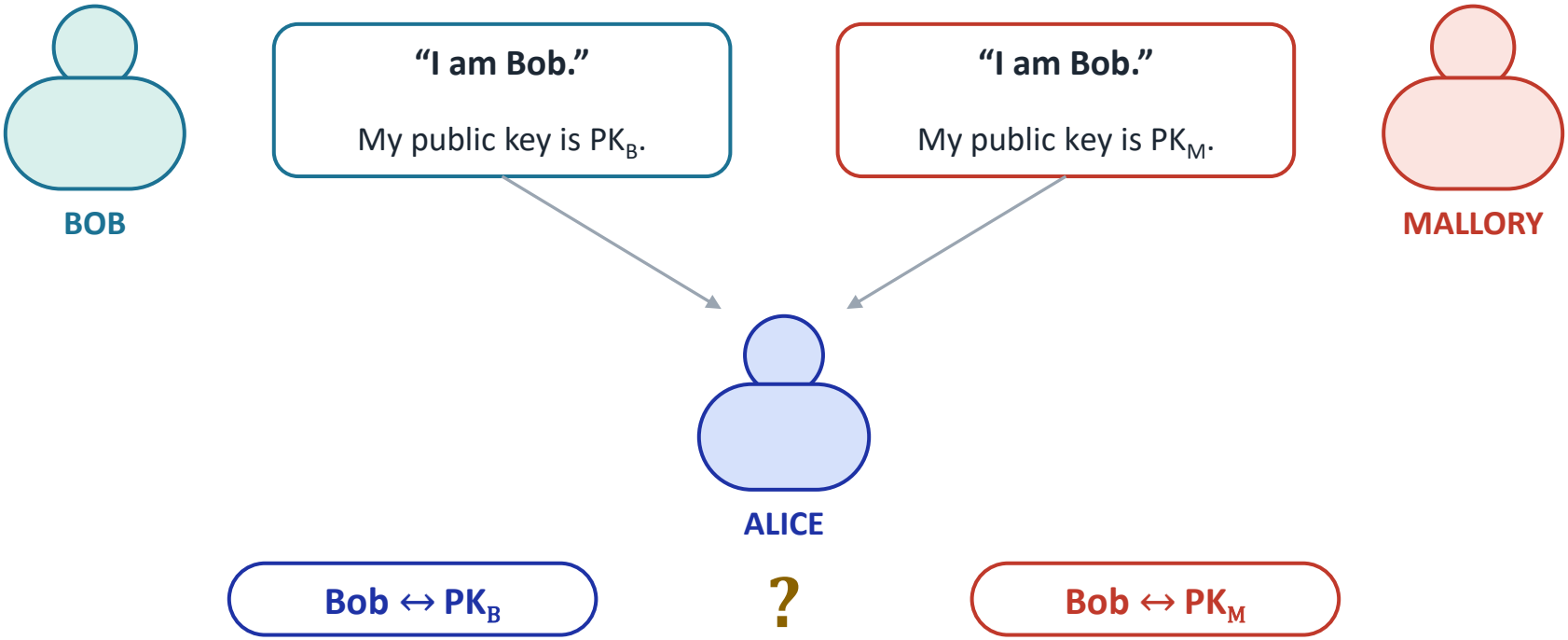
Authentic? ?

? THINK LIKE AN ENGINEER

If public keys are not secret, what could possibly go wrong?

Making a public key public is easy; authenticating who owns it is not.

Mallory Can Say "I Am Bob" Too



Both can be perfectly valid cryptographic keys.

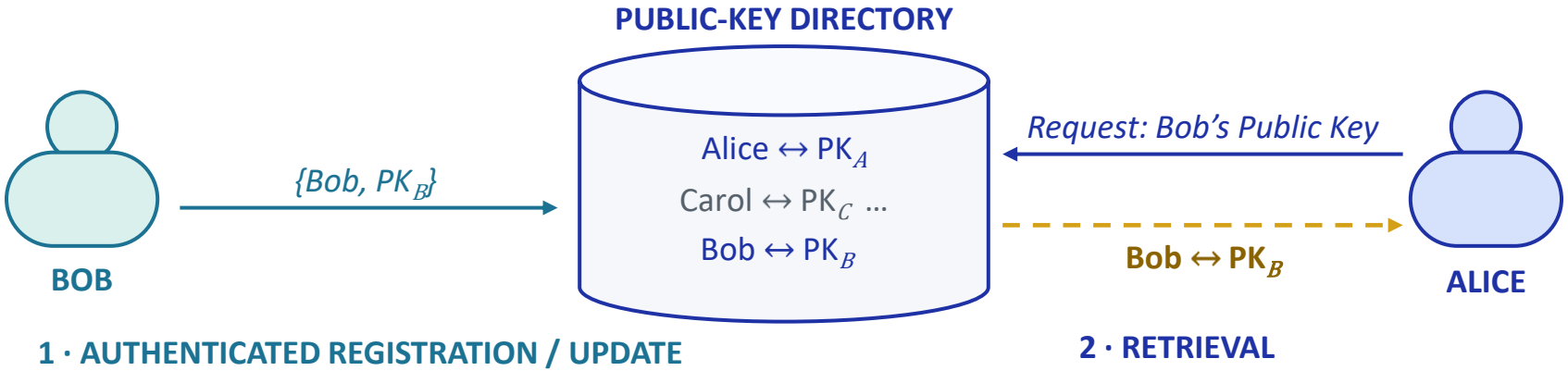
Valid key ≠ valid identity binding

? THINK LIKE AN ENGINEER

What must the next design add that announcement lacks?

A public key needs an authenticated identity binding—not merely a distribution channel.

Put the Identity–Key Bindings in a Trusted Directory



DIRECTORY RESPONSIBILITIES

- maintain {identity, public key} bindings
- authenticate registration / key replacement
- make bindings publicly retrievable

THE PROGRESSION SO FAR

ANNOUNCEMENT → DIRECTORY

"Trust the claimed binding."

"From a controlled repository."

But the retrieval itself still needs trustworthy protection.

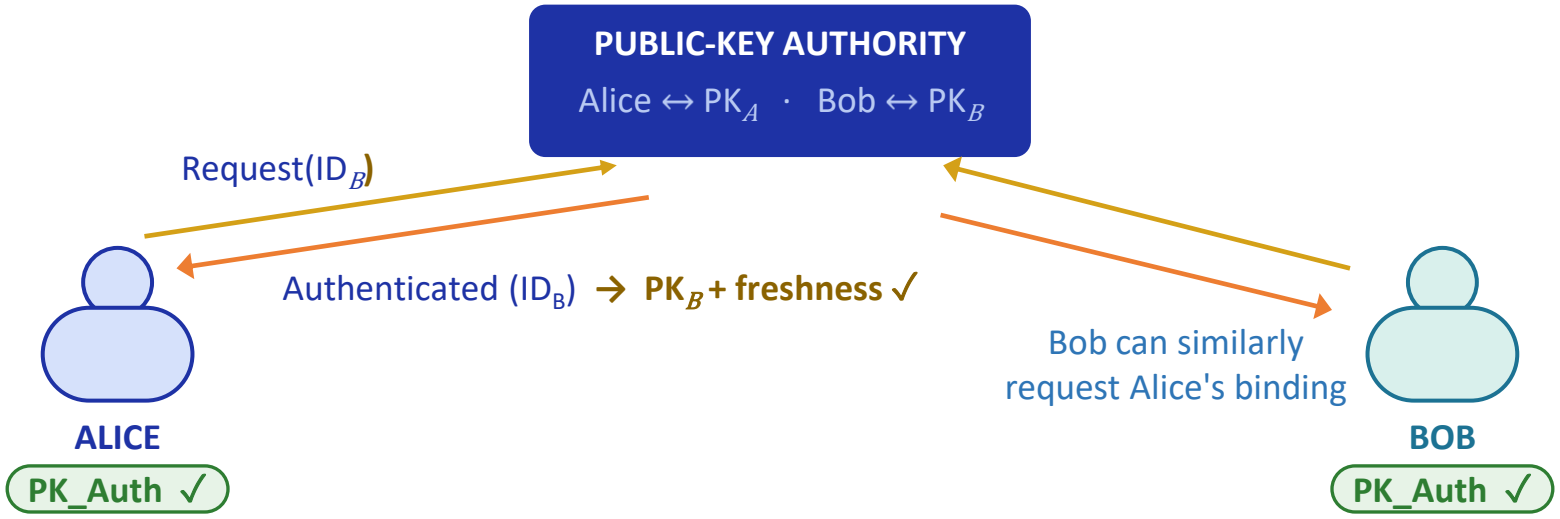
Remaining problem: how does Alice authenticate what she retrieves?

? THINK LIKE AN ENGINEER

Can the directory authenticate its answer to Alice?

The directory controls the binding; its delivery to Alice still needs protection.

Ask a Trusted Authority for the Public Key



Authority's public key is already trusted — everyone can know it; only the Authority holds the matching private key.

DIRECTORY: retrieve stored binding → **AUTHORITY**: retrieve **authenticated + fresh** binding

AUTHENTICATED RESPONSE *protects the reply*	**FRESHNESS** *guards against replay*	**CONTROLLED RETRIEVAL** *through the Authority*

Public keys can be cached and refreshed periodically, once retrieved and authenticated.

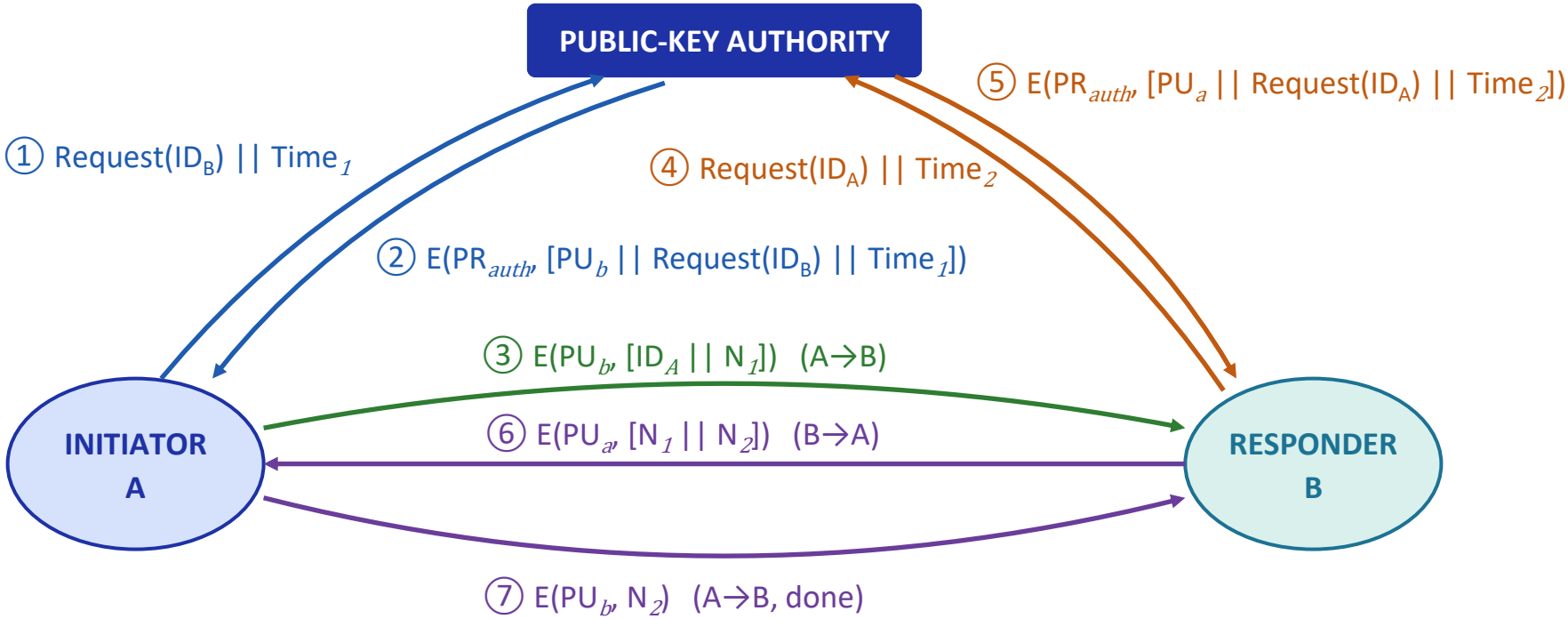
? THINK LIKE AN ENGINEER

Alice can now trust PK_B. What must Bob obtain before he can authenticate Alice?

The Authority authenticates public-key bindings at retrieval time.

PUBLIC-KEY AUTHORITY · PROTOCOL FLOW

How Does the Public-Key Authority Protocol Work?



Msg ③ only claims ID_A — B needs a trusted PU_a before responding.

Request = ask for the peer's current public key · PR_{auth} = Authority's private key · N_1, N_2 = nonces

Authority is on the key-retrieval path — *keys may be cached, but fresh copies must be obtained periodically.*

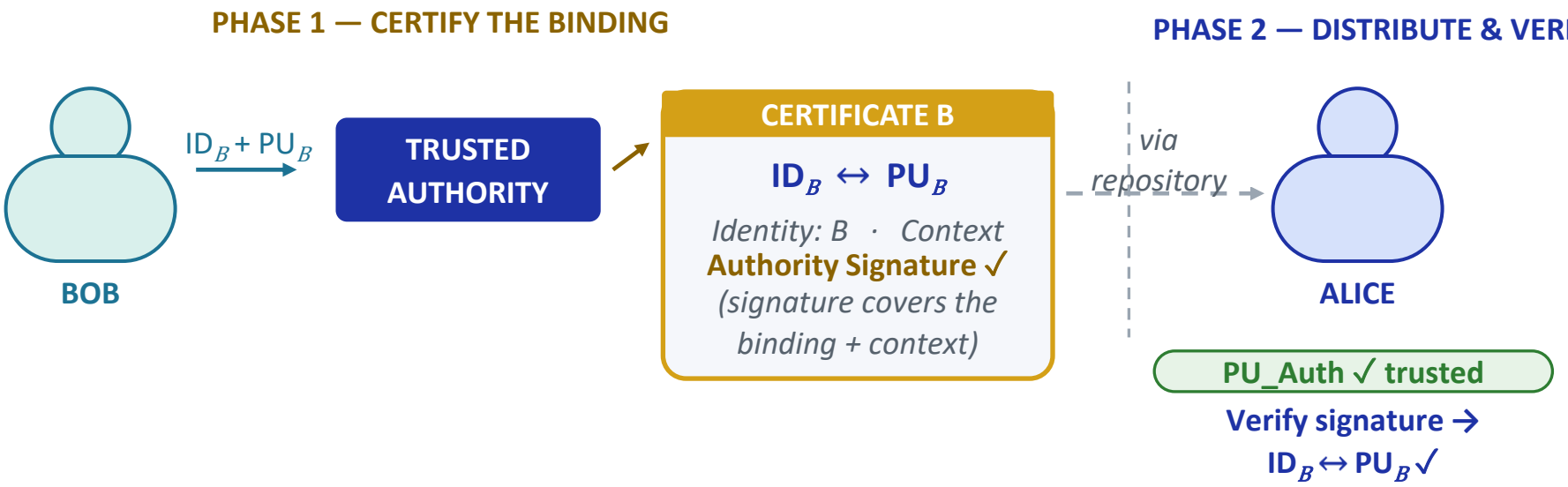
? THINK LIKE AN ENGINEER

Can we keep the trusted identity–key binding without contacting the Authority at retrieval time? (→ certificates, next.)

The Authority authenticates public-key retrieval; A and B then use the trusted keys in a fresh challenge–response exchange.

MEDIATED TRUST · CERTIFICATE

Authenticate the Binding Once — Verify It Later



PUBLIC-KEY AUTHORITY — authenticate at retrieval (Alice → Authority → binding)
CERTIFICATE — authenticate at issuance, verify later (Authority → Certificate → … → Alice)
Authority's trust role has changed — it is no longer on Alice's immediate retrieval path.

WHAT DOES THE CERTIFICATE ESTABLISH? The certificate binds identity information to a public key.
The entity may still need to demonstrate possession of the private key during authentication.

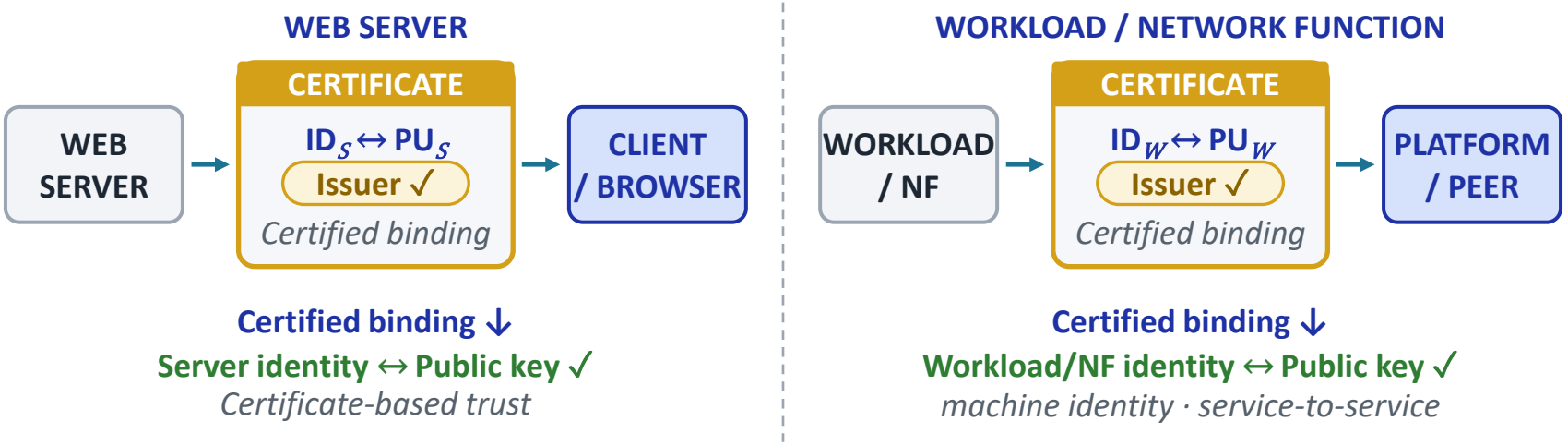
? THINK LIKE AN ENGINEER

Why can Alice verify Bob's binding without contacting the Authority now?

A certificate captures an Authority-authenticated identity–public-key binding for later verification.

APPLIED CONNECTION · CERTIFIED TRUST

Where Do We Use Certified Identity–Key Bindings?



DIFFERENT ENTITIES — SAME TRUST PROBLEM

Who does this public key belong to?

Web server identity ↔ Public key

Workload/NF identity ↔ Public key

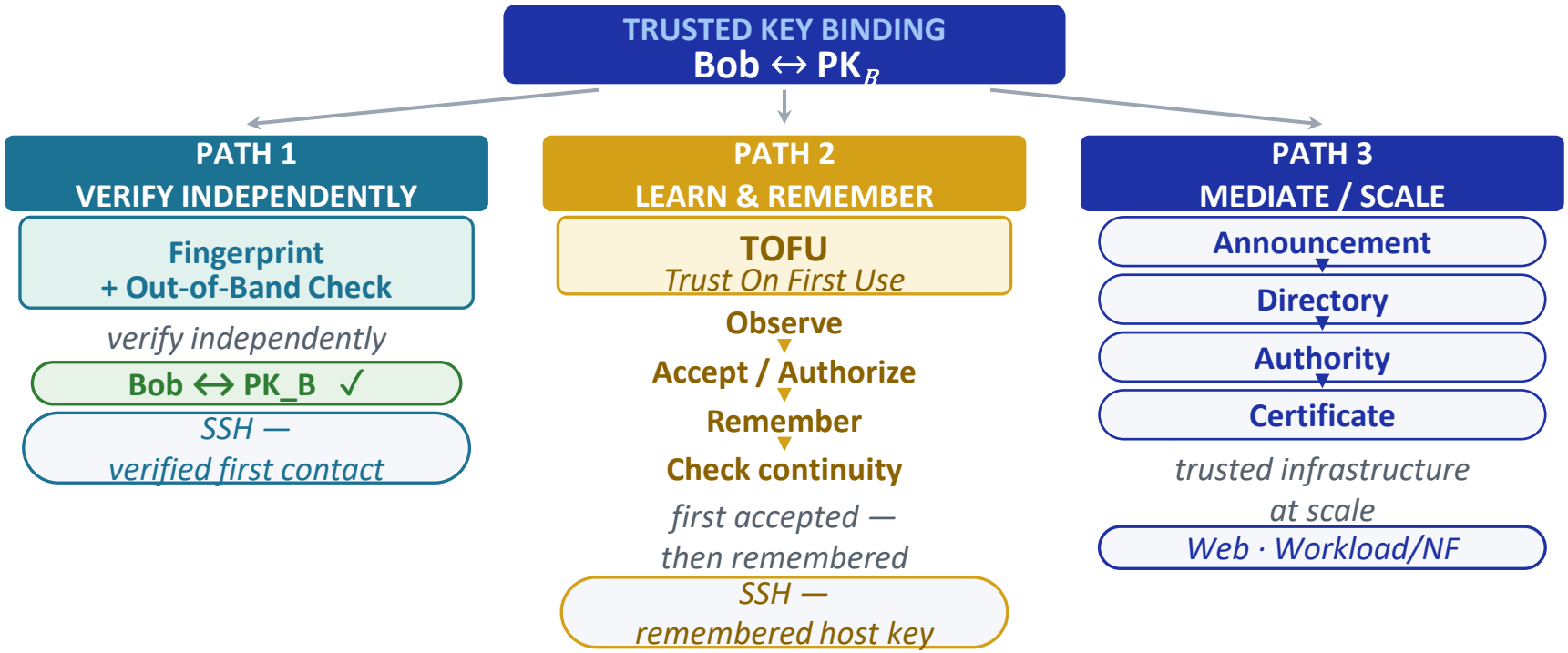
? THINK LIKE AN ENGINEER

What is common between authenticating a web server and onboarding a network function?

Certificates extend trusted public-key bindings to servers, services and network functions.

SYNTHESIS · TRUST MODELS

Different Paths — The Same Trust Question



Direct verification → Remembered trust → Mediated trust at scale

1 · A mathematically valid public key does not establish whose key it is.
2 · Trust models provide different evidence or assumptions for accepting an identity ↔ public-key binding.
3 · The appropriate model depends on assurance needs, infrastructure, scale and first-contact assumptions.

1 · Where does the trusted binding come from?
2 · How is the binding authenticated or accepted?
3 · What happens when the key changes?

NEXT — *How does X.509 represent and validate a certificate binding?*

Public-key security depends not only on the key — but on why we trust who owns it.

REFERENCES · FURTHER READING

Public-Key Distribution and Trust Models — References

TEXTBOOK FOUNDATION

William Stallings

Cryptography & Network Security — §15.3

Announcement | Directory | Public-Key Authority | Certificates

Primary source for this lecture's models

Christof Paar & Jan Pelzl

Understanding Cryptography — Ch.14 §14.4

Key management | Asymmetric techniques | Certificates

DEPLOYED PROTOCOL

IETF RFC 4251 — SSH Protocol Architecture

§4.1 — Host Keys

SSH host keys | Stored host-key bindings | External verification

IETF RFC 4255 — SSH Key Fingerprints

Publishing SSH fingerprints via DNS

Host-key fingerprints | First contact | OOB verification

Supports the applied SSH discussion

KEY-MANAGEMENT GUIDANCE

NIST SP 800-57 Part 3 Rev. 1

Application-Specific Key Management Guidance

SSH | Host-key verification | Application-specific key management

Supplementary guidance, not the primary source

Classical models → Stallings *Key mgmt/certs → Paar & Pelzl* *OOB / SSH practice → IETF + NIST*

Textbooks establish the models; standards show how trust decisions appear in deployed systems.